Practical Multilayer Network Analysis with Py3plex

Release 1.1.5

Blaž Škrlj

Mar 23, 2026

CONTENTS

1 Front Matter 1

2 Introduction: Why Multilayer Modeling Changes Results 3

3 Multilayer Basics: Semantics Before Computation 5

4 Design of py3plex: Trade-offs and Boundaries 7

5 Installation and First Verified Run 9

6 Data Loading and Representation Choices 11

7 Visualization as Diagnostic Analysis 13

8 Core Algorithms: What Is Estimated, What Is Approximated 15

9 DSL Fundamentals: A Query Mental Model 17

10 Builder API and Explain Plans: Reading Your Own Query 19

11 Advanced Workflows: Compositional Queries Under Uncertainty 21

12 Limitations and Stability: What py3plex Does Not Promise 23

13 Case Study 1 — Social Multiplex: Influence vs Brokerage 25

14 Case Study 2 — Biological Multilayer: Robust Signal or Layer Artifact? 27

15 Case Study 3 — Transportation Network: Resilience Under Layer Disruption 29

16 Testing and Validation as Methodological Controls 31

17 Reproducible Environments and Replayable Analysis 33

18 GUI Overview: Where It Helps, Where It Does Not 35

19 Appendix A: Repository Layout and Scripts 36

20 Appendix B: Docker, Docker Compose, and Deployment 40

21 Appendix C: Detailed Validation Scripts 49

22 Appendix D: Error Handling and Exception Hierarchy 56

23 Appendix E: Extended API and DSL Reference **68**

24 Bibliography **76**

Bibliography **79**

CHAPTER

ONE

FRONT MATTER

1.1 About This Book

Practical Multilayer Network Analysis with Py3plex is a technical handbook for readers who already know basic graph analytics and want to make better modeling decisions on multilayer data.

This is not a complete API catalog. It is a working text about representation, inference, failure modes, and reproducibility.

1.2 Scope and Reader Contract

The core narrative is organized around four questions:

1. When does multilayer modeling change the scientific conclusion?
2. Which py3plex implementation choices are consequential for that conclusion?
3. Which approximations are acceptable under realistic constraints?
4. How do we make the workflow auditable and reproducible?

The book assumes intermediate Python fluency, familiarity with standard graph concepts, and comfort reading short mathematical arguments.

1.3 How to Read This Book

- **Part I (Foundations)** defines semantics and design boundaries.
- **Part II (Working Practice)** covers onboarding, representation choices, visualization, and algorithm families.
- **Part III (DSL)** teaches query reasoning, not just syntax.
- **Part IV (Case Studies)** demonstrates analytical payoff and contestable modeling choices.
- **Part V (Systems)** treats testing and reproducibility as central scientific practice.

Reference-heavy material (deployment details, long validation scripts, API listings) is placed in appendices to keep the main text focused.

1.4 What This Book Deliberately Does Not Do

- It does not claim that multilayer modeling is always superior.
- It does not treat convenience interfaces as methodological guarantees.
- It does not equate reproducible code execution with valid scientific inference.

1.5 About the Software

This edition targets **py3plex 1.1.5**. py3plex implements multilayer data structures, algorithm wrappers, DSL tooling, and workflow utilities. Some operations delegate to single-layer backends after explicit transformations; those transitions are identified in the relevant chapters.

1.6 Conventions

- Code blocks are minimal and intended to expose analytical decisions.
- We label whether a statement is a **concept**, **implementation choice**, **approximation**, or **workflow recommendation**.
- Caveats are presented inline where methods are introduced.

1.7 Acknowledgment and Citation

If py3plex contributes to published work, cite:

```
@Article{Skrlj2019,
  author={Skrlj, Blaz and Kralj, Jan and Lavrac, Nada},
  title={Py3plex toolkit for visualization and analysis of multilayer networks},
  journal={Applied Network Science},
  year={2019},
  volume={4},
  number={1},
  pages={94},
  doi={10.1007/s41109-019-0203-7}
}
```

1.8 License

Note

Version Information: This book edition is version 1.1.5 (2025), aligned with py3plex 1.1.5 and Python 3.8+ runtime support.

CHAPTER

TWO

INTRODUCTION: WHY MULTILAYER MODELING CHANGES RESULTS

Most network tutorials begin with capabilities. This chapter begins with failure: what single-layer analysis gets wrong when relationships are heterogeneous.

2.1 Problem Framing

In many domains, edges carry incompatible semantics:

- social ties mix trust, co-work, and kinship,
- transport links mix modes with different costs and failure dynamics,
- biological links mix physical interaction, regulation, and co-expression.

Flattening these into one graph is not just information loss. It changes the estimand. A centrality score in a flattened graph answers a different question than a score on a structured multilayer representation.

2.2 A Running Contrast: Flattened vs Multilayer

Consider a commuter system with rail and bus layers:

- In a flattened graph, high transfer stations can dominate centrality by construction.
- In a multilayer model, we can separate within-mode influence from transfer dependence.

The practical consequence is interpretive: "important node" may mean resilient hub, fragile bottleneck, or mode-bridging artifact depending on representation.

2.3 What py3plex Contributes (and What It Does Not)

py3plex provides a multilayer data model and analysis interfaces that make these distinctions executable in Python workflows.

It does **not** remove modeling judgment. You still choose:

- how layers are defined,
- what inter-layer edges mean,
- whether approximation is acceptable,
- which uncertainty quantification method is relevant.

These choices are methodological, not merely software settings.

2.4 Three Questions to Carry Through the Book

1. **Representation:** Is a layer boundary meaningful, or only convenient?
2. **Inference:** Does an algorithm estimate what we think it estimates under this representation?
3. **Credibility:** Are the results stable under perturbation, alternative parameterizations, and reproducibility constraints?

2.5 Audience and Prerequisites

This text is aimed at graduate researchers and practitioners who already use graph analytics and now need multilayer rigor. We assume:

- Python proficiency,
- familiarity with basic graph metrics,
- willingness to inspect assumptions rather than accept defaults.

2.6 Chapter Roadmap

- Chapter 2 formalizes multilayer semantics and common traps.
- Chapter 3 explains py3plex design trade-offs.
- Part II moves to practical workflows.
- Part III focuses on DSL reasoning.
- Part IV turns methods into domain arguments through case studies.

CHAPTER

THREE

MULTILAYER BASICS: SEMANTICS BEFORE COMPUTATION

This chapter defines multilayer network objects and highlights semantic mistakes that produce misleading analyses.

3.1 Formal Object

A multilayer network can be represented as $\mathcal{M} = (V, L, V_M, E_{intra}, E_{inter})$, where:

- V is the set of physical entities,
- L is the set of layers,
- $V_M \subseteq V \times L$ is the set of node replicas,
- E_{intra} links replicas within a layer,
- E_{inter} links replicas across layers.

3.2 Concept vs Implementation

Concept: node replicas are analytical units that encode context.

py3plex implementation: nodes are stored as `(node, layer)` pairs in the core graph, so many operations naturally return replica-level results.

Do not silently reinterpret replica counts as physical-node counts.

3.3 Replica Semantics: The First Major Pitfall

If a physical node appears in three layers, it has three replicas. This matters for:

- counts,
- degree interpretation,
- top-k selection,
- community assignments.

A frequent novice error is to compute top hubs globally, then report them as physical entities without de-duplicating by base node.

3.4 Degree Is Ambiguous in Multilayer Contexts

At least three notions of degree can be relevant:

- **intra-layer degree** (within one layer),
- **inter-layer degree** (coupling/transfer links),
- **aggregate degree** (combined).

In py3plex workflows, aggregate degree is often the default unless per-layer operations are explicitly applied.

Interpretive warning: aggregate degree is not necessarily a better statistic; it is a different statistic.

3.5 Coverage Semantics and False Certainty

Coverage filters encode cross-group logic:

- `all`: intersection,
- `any`: union,
- thresholded modes (`at_least`, `fraction`): partial overlap.

`all` is strict and can yield very small result sets. This is often interpreted as "only a few robust nodes," but it may instead reflect harsh filtering under high layer heterogeneity.

3.6 Global vs Per-Layer Operations

Two analyses can be correct yet answer different questions:

- **global** community detection: seeks communities spanning layers,
- **per-layer** detection: compares community structure between layers.

Neither is universally preferred. Choose based on hypothesis, not convenience.

3.7 Minimal Sanity Checklist

Before trusting any multilayer output:

1. Confirm whether reported units are replicas or physical nodes.
2. State which degree semantics are used.
3. State whether operations were global or grouped by layer.
4. Report coverage mode and thresholds.
5. Provide at least one alternative analysis path as a robustness check.

3.8 Why This Chapter Matters

Most downstream errors are not coding errors. They are semantic errors introduced before algorithm choice. Getting these distinctions right is the main precondition for meaningful multilayer inference.

CHAPTER

FOUR

DESIGN OF PY3PLEX: TRADE-OFFS AND BOUNDARIES

This chapter explains why py3plex is structured the way it is, and what those choices imply for analysis quality.

4.1 Design Goal

py3plex is designed to make multilayer analysis practical in Python without hiding key modeling decisions.

The project prioritizes:

1. a consistent multilayer representation,
2. interoperability with established graph tooling,
3. explicit caveats around approximations and feature maturity.

4.2 Core Architectural Choice

py3plex builds on NetworkX-compatible graph objects while layering multilayer semantics on top.

Benefit: immediate access to mature single-layer algorithms and ecosystem tooling.

Cost: some multilayer operations necessarily pass through reductions (for example, layer-restricted or projected computations). Those transitions can alter interpretation if not documented.

4.3 Theory, Implementation, Workflow

Throughout py3plex, it helps to separate three categories:

- **Theory:** what a multilayer statistic means mathematically.
- **Implementation:** how py3plex computes or delegates that statistic.
- **Workflow:** how analysts sequence filtering, computation, and validation in practice.

Confusing these categories leads to overclaiming. An implementation convenience is not theoretical justification.

4.4 What py3plex Optimizes For

- reproducible scripting workflows,
- incremental exploration from small to medium-scale multilayer graphs,
- composable query pipelines (especially via DSL in Part III).

4.5 What py3plex Does Not Optimize For

- maximal performance on very large graphs without approximation,
- complete replacement of specialized HPC graph libraries,
- automatic protection from poor modeling assumptions.

4.6 Implications for Users

If your analysis is sensitive to representation choices or approximation error:

1. record provenance and seeds,
2. run at least one robustness check,
3. compare multilayer and flattened baselines explicitly,
4. report implementation path (native multilayer vs delegated/projection path).

4.7 Why This Matters

Architecture is methodology in practice. Knowing where py3plex delegates, approximates, or preserves multilayer semantics is part of responsible interpretation.

CHAPTER

FIVE

INSTALLATION AND FIRST VERIFIED RUN

This chapter gives one supported onboarding path. Alternatives and deployment-heavy setups are in Appendix B.

5.1 Golden-Path Setup

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\\Scripts\\activate
pip install --upgrade pip
pip install py3plex
```

If you need editable development mode, install from source in a separate environment. Keep research and development environments distinct.

5.2 Minimal Smoke Test

```
from py3plex.core import multinet
from py3plex.dsl import Q

net = multinet.multi_layer_network(directed=False)
net.add_edges([
    ['Alice', 'social', 'Bob', 'social', 1],
    ['Bob', 'social', 'Cara', 'social', 1],
], input_type='list')

result = Q.nodes().compute('degree').execute(net)
print(result.count)
```

Interpretation note: this verifies installation and execution path only. It does not validate model semantics.

5.3 First-Run Checks That Actually Matter

After the smoke test:

1. verify your Python version and py3plex version,
2. verify deterministic behavior by setting explicit seeds in stochastic workflows,
3. confirm that your intended file formats are parsed as expected.

5.4 Common Early Failures

- **Import succeeds, query fails:** often missing optional dependencies for specific algorithms.
- **Unexpected node counts:** replica vs physical-node confusion.
- **Platform mismatch in scripts:** shell-activation differences or path assumptions.

Do not debug these by adding ad-hoc hacks to notebooks; fix environment specification first.

5.5 Where Extended Setup Lives

For Docker, CI images, and deployment-oriented setup, use Appendix B. Keeping those details out of the core narrative prevents onboarding chapters from becoming support manuals.

CHAPTER

SIX

DATA LOADING AND REPRESENTATION CHOICES

Loading data is not a clerical step. Representation decisions made here determine what your metrics can mean later.

6.1 Representation Decisions with Analytical Consequences

Before importing anything, answer:

1. What constitutes a layer in this domain?
2. Are inter-layer edges observed data, inferred links, or modeling conveniences?
3. Are edge weights comparable across layers?
4. What entity identity rules resolve duplicates and missingness?

These decisions should be documented alongside code.

6.2 A Minimal, Auditable Loading Pattern

```
from py3plex.core import multinet

net = multinet.multi_layer_network(directed=False)
net.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Alice', 'type': 'work'},
    {'source': 'Bob',   'type': 'social'},
])
net.add_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'social', 'target_type': 'social'},
])

layers = net.get_layers()
print(layers)
```

Implementation detail: py3plex expects explicit layer attributes in node and edge dictionaries for multilayer semantics.

6.3 Format Selection (Practical Rule)

Use format choice as a workflow decision:

- **CSV/edgelist:** transparent and easy to diff; good for iterative cleaning.

- **GraphML/GML:** richer metadata, stronger interoperability.
- **Arrow/Parquet pipelines:** useful when throughput and schema stability matter.

No format is universally best. Prefer the one that minimizes ambiguity in your current pipeline.

6.4 Validation Before Analysis

At minimum, validate:

- layer names and counts,
- missing node or layer labels,
- self-loops and duplicate edges (if relevant to your methods),
- weight domain assumptions (non-negative, normalized, etc.).

A naive mistake is to accept parser success as data validity.

6.5 What Can Go Wrong

- Layer labels encoded inconsistently (e.g., *Social*, *social*, *soc*).
- Coupling edges mixed with domain edges without tagging.
- Flattened imports accidentally treated as multilayer outputs.

These errors usually survive until interpretation, where they are expensive to detect.

6.6 Recommendation

Treat data loading code as part of your methodological appendix. If another analyst cannot reconstruct your representation choices from that code, the pipeline is not yet ready.

CHAPTER

SEVEN

VISUALIZATION AS DIAGNOSTIC ANALYSIS

Visualization in multilayer work is best used as a diagnostic instrument, not as proof by picture.

7.1 Analytical Purpose of Plots

Use visualization to test hypotheses about structure:

- Are layers genuinely distinct or mostly redundant?
- Are bridging nodes meaningful connectors or layout artifacts?
- Are inter-layer links interpretable under the domain model?

If the plot cannot answer one of these questions, it is likely decorative.

7.2 A Focused Workflow

```
from py3plex.visualization.multilayer import draw_multilayer_default

layers, layer_graphs, positions = network.get_layers()
draw_multilayer_default((layers, layer_graphs, positions), display=False)
```

Implementation note: layout defaults are convenience choices. They are not statistical estimators.

7.3 Common Misreadings

1. **Large central node = important actor** Often false when size encodes aggregate degree across incomparable layers.
2. **Dense layer = stronger system component** May reflect data collection bias or thresholding rules.
3. **Clean separation = true modularity** May be a force-layout artifact.

7.4 How to Make Plots More Trustworthy

- annotate what is encoded by size, color, and edge opacity,
- compare at least two layout seeds for stability,
- pair visual claims with numeric checks (coverage, centrality summaries, modularity diagnostics),
- keep layer-specific and global plots side-by-side.

7.5 When Not to Rely on Visualization

For very large graphs, visual clutter makes node-level interpretation unreliable. Prefer summary statistics and targeted subgraph diagnostics.

7.6 Conclusion

In this book, visualizations are exploratory and communicative aids. Claims should be grounded in reproducible computations, with plots used to expose potential structure and potential misunderstanding.

CHAPTER

EIGHT

CORE ALGORITHMS: WHAT IS ESTIMATED, WHAT IS APPROXIMATED

This chapter organizes py3plex algorithm use around interpretation risk rather than interface breadth.

8.1 Three Families

1. **Community detection** (partition structure)
2. **Centrality** (node or edge importance under specific definitions)
3. **Dynamics** (state evolution under explicit process assumptions)

8.2 Community Detection

py3plex supports multilayer community workflows through methods such as Louvain/Leiden-style procedures and related wrappers.

Interpretive caution:

- Partition quality metrics are objective-specific.
- Comparable scores do not imply identical community semantics across methods.
- Global vs per-layer community detection answer different questions.

8.3 Centrality

Many centrality measures are available, but users must state:

- whether analysis is per-layer or global,
- whether values are exact or approximated,
- whether measures were computed on native multilayer structure or reduced projections.

Approximation is often suitable for ranking-oriented exploration on larger graphs, but claims about small score differences should be avoided unless validated.

8.4 Dynamics

SIS/SIR-like models in py3plex are useful for scenario analysis under explicit assumptions.

Do not infer causal epidemiological truths from toy parameter sweeps. Dynamics outputs are model-conditional, not domain truth.

8.5 Practical Pattern

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['social'])
     .compute('degree', 'betweenness_centrality')
     .order_by('-degree')
     .limit(20)
     .execute(network)
)
```

What this gives: a ranked summary under a specific representation.

What it does not give: robustness, causal explanation, or cross-representation invariance.

8.6 Method Selection Checklist

Before choosing an algorithm, specify:

1. target estimand,
2. acceptable approximation error,
3. computational budget,
4. validation strategy (alternative methods, perturbation, or UQ).

This prevents the common workflow error of selecting methods by convenience rather than inferential fit.

CHAPTER

NINE

DSL FUNDAMENTALS: A QUERY MENTAL MODEL

The DSL is useful when you need clear, replayable analytical pipelines over multilayer data. This chapter teaches the mental model first and syntax second.

9.1 Why a DSL Here?

A query pipeline makes assumptions explicit:

- selection scope (which layers, which entities),
- computed measures,
- ordering and filtering logic,
- reproducibility settings.

This helps analysts review and reproduce analytical intent.

9.2 Core Workflow Pattern

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['social'])
     .where(degree__gt=3)
     .compute('betweenness_centrality')
     .order_by('-betweenness_centrality')
     .limit(10)
     .execute(network)
)
```

Interpretation boundary: this is a reproducible computational selection, not automatically a defensible scientific claim.

9.3 Theory vs Implementation vs Workflow

- **Theory:** filtering by degree defines a set under a degree function.
- **Implementation:** py3plex resolves fields, may autocompute metrics, and executes against internal graph structures.
- **Workflow:** your chosen thresholds and ordering reflect domain judgment.

Keep these separate in analysis reports.

9.4 Layer Algebra and Scope Control

Layer selection should reflect hypothesis scope, not convenience:

```
from py3plex.dsl import L

social_or_work = L['social'] + L['work']
social_not_work = L['social'] - L['work']
```

A common error is using all layers by default and inferring layer-specific conclusions.

9.5 What New Users Usually Misunderstand

1. Query fluency does not remove semantic ambiguity (replicas vs physical nodes).
2. A short query can still encode strong assumptions.
3. Ranking stability requires separate checks (UQ, sensitivity, or perturbation).

9.6 Next Step

Chapter 9 covers builder internals and explain plans. Chapter 10 covers advanced workflows where query complexity can hide methodological risk.

CHAPTER

TEN

BUILDER API AND EXPLAIN PLANS: READING YOUR OWN QUERY

This chapter focuses on query structure, execution traceability, and failure diagnosis.

10.1 Builder as Structured Intent

The builder API is most valuable when it makes intent reviewable by someone other than the original author.

A useful pattern is to keep pipelines in named stages:

```
from py3plex.dsl import Q, L

scoped = Q.nodes().from_layers(L['social'])
filtered = scoped.where(degree__gt=5)
measured = filtered.compute('degree', 'pagerank')
ranked = measured.order_by('-pagerank').limit(20)
result = ranked.execute(network)
```

This is less concise than one long chain, but easier to audit.

10.2 Explain Plans

Use explain tooling to inspect what will be executed and in what order. This is especially important when autocompute, grouping, or coverage steps are involved.

Ask three questions when reading a plan:

1. Which fields are materialized and when?
2. Which operations depend on grouped context?
3. Where could expensive computations be delayed or reduced?

10.3 Parameterized Queries

For repeated analyses, parameterize thresholds rather than editing literals in notebooks.

```
from py3plex.dsl import Q, Param

query = Q.nodes().where(degree__gt=Param.int('k')).compute('degree')
result = query.execute(network, k=4)
```

This improves reproducibility and reduces accidental drift across runs.

10.4 Failure Diagnostics

When queries fail, categorize the issue quickly:

- syntax-level problem,
- unknown field/measure,
- grouping misuse,
- missing parameters,
- type mismatch.

Treat diagnostic messages as part of your workflow review process, not only as debugging output.

10.5 When the Builder Is Not the Right Tool

If your workflow is a one-off simple filter, direct Python operations may be clearer. DSL helps most when you need replayability, composability, and explicit provenance.

CHAPTER

ELEVEN

ADVANCED WORKFLOWS: COMPOSITIONAL QUERIES UNDER UNCERTAINTY

Advanced DSL usage is not about longer chains. It is about controlling semantic scope, computational cost, and uncertainty propagation.

11.1 Workflow Pattern 1: Grouped Selection with Coverage

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['*'])
     .compute('degree')
     .per_layer()
       .top_k(10, 'degree')
     .end_grouping()
     .coverage(mode='at_least', k=2)
     .execute(network)
)
```

Judgment point: coverage threshold controls strictness. `all` can be too restrictive in heterogeneous systems.

11.2 Workflow Pattern 2: UQ-Aware Ranking

```
result = (
    Q.nodes()
     .compute('pagerank')
     .uq(method='bootstrap', n_samples=100, seed=42)
     .order_by('-pagerank')
     .limit(20)
     .execute(network)
)
```

Interpretive warning: confidence intervals quantify algorithm/data perturbation under a model; they do not validate domain truth.

11.3 Workflow Pattern 3: Temporal Slicing

```
temporal = (
    Q.edges()
     .during(100.0, 150.0)
     .from_layers(L['transport'])
     .execute(temporal_network)
)
```

Judgment point: time-window boundaries can dominate observed effects. Report window design choices explicitly.

11.4 Composability vs Readability

As pipelines grow, readability declines. Recommended practices:

- split complex pipelines into named stages,
- log final query strings or AST hashes,
- avoid mixing exploratory and production logic in one chain.

11.5 Performance and Approximation

For larger networks:

- use selective layer scopes before expensive measures,
- use approximations when ranking is sufficient,
- record approximation parameters in outputs.

Never present approximate outputs as exact without explicit disclosure.

11.6 Advanced Checklist

Before finalizing an advanced query workflow:

1. verify semantic scope (global vs per-layer),
2. verify uncertainty configuration,
3. verify seeds and provenance capture,
4. verify that outputs answer the intended question.

CHAPTER

TWELVE

LIMITATIONS AND STABILITY: WHAT PY3PLEX DOES NOT PROMISE

This chapter is intentionally strict. It defines reliability boundaries for methods used in this book.

12.1 Scope of Stability Statements

A statement like "stable API" means the workflows documented here are intended to remain usable across minor revisions. It does **not** mean every internal behavior is immutable.

12.2 Major Limitation Classes

1. **Semantic limitations** Multilayer outputs can be misread if replica-level and physical-node interpretations are mixed.
2. **Algorithmic limitations** Some methods rely on assumptions (connectivity, weight domains, stochastic convergence) that may fail silently if not checked.
3. **Scalability limitations** Exact methods can become impractical on larger graphs; approximations trade precision for tractability.
4. **Interoperability limitations** Delegation to single-layer backends can alter semantics if projection steps are not explicit.

12.3 Stability Practices That Help

- pin versions for reproducible studies,
- record query provenance and random seeds,
- include at least one robustness comparison,
- keep method caveats near reported results.

12.4 What to Report in Publications

At minimum, report:

- representation choices,
- algorithm and parameter settings,
- approximation/UQ configuration,
- software version and environment details,
- known limitations relevant to your claims.

A short, honest limitations paragraph is usually more credible than broad claims of robustness.

CHAPTER

THIRTEEN

CASE STUDY 1 — SOCIAL MULTIPLEX: INFLUENCE VS BROKERAGE

workflow template

This chapter uses a reusable workflow template for social multiplex analysis. The template is intentionally explicit about assumptions so readers can adapt it to their own data.

13.1 Research Question

How does influence ranking change when friendship, collaboration, and mentorship ties are modeled as separate layers instead of a single flattened graph?

13.2 Data and Representation Choices

We represent each person as replicas across three layers:

- `friendship`
- `collaboration`
- `mentorship`

Contestable choice: inter-layer edges connect replicas of the same person with uniform coupling weight. This simplifies identity continuity but may underrepresent context switching costs.

13.3 Baseline: Flattened Analysis

```
flat = network.flatten_to_monoplex(method='union')
# Run standard centrality on flattened graph
```

Flattened ranking highlights well-connected generalists. It cannot distinguish whether influence comes from one dominant layer or cross-layer brokerage.

13.4 Multilayer Analysis

```
from py3plex.dsl import Q, L

per_layer = (
    Q.nodes()
```

```
    .from_layers(L['friendship'] + L['collaboration'] + L['mentorship'])
    .compute('degree', 'betweenness_centrality')
    .per_layer()
      .top_k(15, 'betweenness_centrality')
    .end_grouping()
    .coverage(mode='at_least', k=2)
    .execute(network)
)
```

Key finding: several individuals absent from flattened top-k appear repeatedly as high-brokerage nodes across layers.

13.5 Why Multilayer Changed the Result

Flattening merged semantically different ties and amplified dense collaboration clusters. The multilayer query isolated cross-context connectors that are structurally modest in any single layer but strategically important across layers.

13.6 Fragile Assumptions

1. Uniform inter-layer coupling may overstate identity continuity.
2. Missingness differs by layer (mentorship ties are often underreported).
3. Top-k thresholds are sensitive to layer size imbalance.

13.7 Robustness Checks

```
uq_result = (
    Q.nodes()
     .compute('betweenness_centrality')
     .uq(method='bootstrap', n_samples=100, seed=42)
     .execute(network)
)
```

We treat rank changes within confidence overlap as ambiguous rather than definitive.

13.8 Transferable Lesson

If the practical question concerns cross-context brokerage, flattened centrality is usually a poor proxy. Layer-aware selection with explicit coverage criteria is more informative.

13.9 Local Caveat

This case relies on relatively stable layer definitions. In domains where layer labels are noisy or evolving, the same workflow needs temporal and schema-uncertainty extensions.

CHAPTER

FOURTEEN

CASE STUDY 2 — BIOLOGICAL MULTILAYER: ROBUST SIGNAL OR LAYER ARTIFACT?

workflow template

This chapter follows the same workflow template used throughout Part IV while adapting assumptions to biological networks.

14.1 Research Question

Do high-centrality genes remain high across interaction layers (PPI, regulation, co-expression), or are "hub" claims layer artifacts?

14.2 Representation and Contestable Choices

Layers:

- `ppi` (physical interactions)
- `regulatory`
- `coexpression`

Contestable choice: we treat all edges as unweighted in the baseline, then compare against a weighted sensitivity run.

14.3 Naive Alternative

A flattened analysis can identify global hubs quickly, but it mixes mechanistically different relations and tends to overweight denser layers.

14.4 Layer-Aware Workflow

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['ppi'] + L['regulatory'] + L['coexpression'])
     .compute('degree', 'pagerank', 'betweenness_centrality')
     .per_layer()
       .top_k(25, 'pagerank')
```

```
    .end_grouping()
    .coverage(mode='at_least', k=2)
    .execute(network)
)
```

Interpretation: nodes retained after coverage filtering are less likely to be single-layer artifacts.

14.5 Uncertainty and Stability

```
stable = (
    Q.nodes()
     .compute('pagerank')
     .uq(method='stratified_perturbation', n_samples=80, seed=42)
     .execute(network)
)
```

This estimates ranking stability under structured perturbation. We do not interpret narrow intervals as biological certainty; they quantify model-internal stability only.

14.6 What Flattening Missed

Flattening promoted genes central in co-expression only. Multilayer filtering identified a smaller set with repeated prominence across mechanistically different layers, yielding candidates more consistent with cross-pathway involvement.

14.7 Fragile Assumptions

1. Layer construction pipelines can introduce correlated bias.
2. Edge confidence scores are heterogeneous across sources.
3. Inter-layer coupling choice affects cross-layer persistence claims.

14.8 Reproducibility Practices

- fixed random seeds,
- archived input snapshots,
- recorded query configuration,
- reported software versions and environment metadata.

14.9 Transferable Lesson

In biological multilayer studies, “important gene” claims should usually be phrased as model-conditional and layer-conditional unless cross-layer persistence is demonstrated.

CHAPTER

FIFTEEN

CASE STUDY 3 — TRANSPORTATION NETWORK: RESILIENCE UNDER LAYER DISRUPTION

workflow template

This chapter provides a workflow template for transportation multilayer analysis with an emphasis on assumptions subject to scrutiny and resilience interpretation.

Readers adapting this template should document transfer-cost assumptions, temporal aggregation choices, and missing-data handling before interpreting ranking outputs.

15.1 Research Question

Which stations are critical for multimodal resilience, and how does that answer change when one mode is degraded?

15.2 Representation Choices

Layers:

- `metro`
- `bus`
- `bike`
- `walk_transfer`

Simplifying assumption: transfer edges are modeled with fixed penalty weights, though in practice transfer costs vary by time, congestion, and accessibility.

15.3 Naive Baseline

A flattened shortest-path analysis finds globally short routes but hides mode dependency. It cannot separate "fast because metro exists" from "resilient across mode failures."

15.4 Multilayer Resilience Workflow

```
from py3plex.dsl import Q, L

critical = (
```

```
    Q.nodes()
     .from_layers(L['metro'] + L['bus'] + L['bike'])
     .compute('betweenness_centrality')
     .per_layer()
       .top_k(20, 'betweenness_centrality')
     .end_grouping()
     .coverage(mode='at_least', k=2)
     .execute(network)
)
```

Then run a disruption scenario by removing or down-weighting one layer and recomputing rankings.

15.5 Why Multilayer Changed the Conclusion

Flattened analysis prioritized metro-core stations because it did not account for mode-specific dependency. The multilayer disruption comparison identified secondary interchange nodes that become dominant under bus or metro degradation.

15.6 Fragile Assumptions

1. Temporal aggregation can erase peak-hour fragility.
2. Static transfer penalties may understate accessibility constraints.
3. Missing pedestrian connectivity biases resilience estimates.

15.7 Reproducibility and Auditability

- fixed seeds for any stochastic components,
- explicit scenario definitions (what is removed or perturbed),
- stored query/provenance metadata for each scenario,
- deterministic export of summary tables.

15.8 Transferable Lesson

For transport planning questions about disruption, multilayer scenario analysis is often more informative than flattened efficiency metrics.

15.9 Local Caveat

This workflow is strongest for topology-driven resilience screening. It is not a substitute for full demand, schedule, or behavioral models.

CHAPTER

SIXTEEN

TESTING AND VALIDATION AS METHODOLOGICAL CONTROLS

Testing is not only software hygiene. In analytical pipelines, tests are controls against silent methodological drift (such as changes in representation or approximation defaults that alter results).

16.1 Validation Layers

py3plex workflows benefit from four test layers:

1. **Unit checks** for deterministic core behavior.
2. **Property/metamorphic checks** for invariants under transformations.
3. **Differential checks** across equivalent APIs.
4. **Workflow checks** that validate representative end-to-end analyses.

16.2 Why This Matters for Analysis

A pipeline can execute successfully and still be wrong:

- representation drift changes semantics,
- approximation defaults change outputs,
- query refactors alter grouping logic.

Tests should target these failure modes directly.

16.3 Practical Workflow

- run focused tests near changed analytical components,
- include at least one deterministic seed-based test for stochastic paths,
- preserve small synthetic fixtures with known expected behavior,
- treat regression diffs as analytical review prompts, not only coding errors.

16.4 Example Validation Questions

- Does node/edge count conservation hold after import transformations?
- Do equivalent query formulations return equivalent sets?
- Are ranking changes under perturbation within expected bounds?

- Are provenance fields complete and stable?

16.5 Connection to Reproducibility

Testing catches accidental changes; reproducibility practices make intentional changes auditable. Both are required for credible technical results.

CHAPTER

SEVENTEEN

REPRODUCIBLE ENVIRONMENTS AND REPLAYABLE ANALYSIS

Reproducibility is an analytical requirement: independent reviewers must be able to re-run your workflow and verify the assumptions made and parameters used in the analysis.

For basic environment setup commands, see Chapter 4. For container/deployment detail, see Appendix B.

17.1 Reproducibility Stack

A reproducible py3plex study should preserve:

1. dependency versions,
2. data snapshots or immutable references,
3. query definitions and parameters,
4. random seeds,
5. execution metadata/provenance.

17.2 Minimal Practice Pattern

- pin package versions in environment files,
- record py3plex version and Python version in outputs,
- store query text/AST and parameter bindings,
- store seeds for stochastic methods,
- save result artifacts with timestamps and scenario labels.

17.3 What Reproducibility Does Not Guarantee

A replayable pipeline can still encode weak assumptions. Reproducibility supports auditability; it does not prove substantive validity.

17.4 Common Failure Modes

- hidden notebook state,
- unpinned dependencies,
- overwritten intermediate files,
- undocumented manual edits,

- inconsistent data extraction windows.

17.5 Mitigation

- use scriptable pipelines over ad-hoc notebook-only workflows,
- keep immutable raw inputs separate from derived data,
- include an explicit run manifest with versions, seeds, and hashes.

17.6 Link to Scientific Credibility

In multilayer analysis, representation and parameter choices are often contestable. Reproducibility makes those choices inspectable and debateable rather than opaque.

CHAPTER

EIGHTEEN

GUI OVERVIEW: WHERE IT HELPS, WHERE IT DOES NOT

The py3plex GUI is a convenience interface for interactive exploration, useful for rapid inspection and teaching workflows. It should not replace versioned analytical scripts.

Status: Experimental

The GUI is intended for local or controlled environments. It is not a hardened public deployment target.

18.1 Appropriate Uses

- quick dataset inspection,
- exploratory layer browsing,
- demonstration and teaching,
- rapid hypothesis sketching before scripted analysis.

18.2 Inappropriate Uses

- sole source of publication-critical results,
- unversioned “click-through” analysis without provenance,
- security-sensitive internet-facing deployment.

18.3 Recommended Practice

Use the GUI to discover questions, then transfer finalized analysis into scriptable py3plex workflows with explicit parameters, seeds, and exports.

This keeps interactive exploration and reproducible inference aligned rather than conflated.

CHAPTER

NINETEEN

APPENDIX A: REPOSITORY LAYOUT AND SCRIPTS

This appendix provides a reference for the py3plex repository structure and maps book examples to actual scripts.

19.1 Repository Structure

19.1.1 Top-Level Organization

```
py3plex/
├── py3plex/              # Main package
├── tests/                # Test suite
├── examples/             # Example scripts (see section below for current layout)
├── docfiles/             # Documentation source (Sphinx)
├── book/                 # This book's source files
├── gui/                  # Web GUI application
├── benchmarks/           # Performance benchmarks
├── datasets/             # Sample datasets
├── pyproject.toml        # Package configuration
├── Makefile              # Development automation
└── README.md             # Project overview
```

19.1.2 Main Package Structure

```
py3plex/
├── __init__.py
├── core/                 # Data structures
│   ├── multinet.py       # multi_layer_network class
│   └── random_generators.py
├── algorithms/           # Analysis algorithms
│   ├── statistics/
│   ├── centrality/
│   ├── community_detection/
│   └── paths/
├── dynamics/             # Dynamical processes
│   ├── models.py         # SIS, SIR, SEIR
│   ├── core.py
│   └── compartmental.py
├── dsl/                  # Query language
│   ├── __init__.py
│   ├── builder.py        # Builder API
│   ├── executor.py
```

```
│   └── export.py
├── visualization/        # Plotting
├── io/                   # I/O handlers
│   ├── readers.py
│   └── writers.py
├── cli.py                # Command-line interface
├── graph_ops.py          # Dplyr-style API
├── pipeline.py           # sklearn-style pipelines
└── workflows.py          # Config-driven workflows
```

19.2 Examples Directory Structure

The `examples/` directory currently contains topic-oriented subdirectories. The exact file count changes over time, so treat this appendix as a navigation map rather than a fixed inventory.

```
examples/
├── getting_started/
├── network_analysis/
├── dsl_zoo/
├── io_and_data/
├── visualization/
├── pipelines/
├── out_of_core/
└── advanced/
```

For current counts and file-level taxonomy, inspect the repository directly (for example: `find examples -maxdepth 2 -type d`).

19.3 Mapping Book Chapters to Examples

19.3.1 Installation and First Verified Run (Installation and Getting Started)

- `examples/getting_started/` — Introductory quickstart scripts

19.3.2 Data Loading and Representation Choices (Data Loading)

- `examples/io_and_data/` — Data loading and conversion workflows

19.3.3 Core Algorithms: What Is Estimated, What Is Approximated (Core Algorithms)

- `examples/network_analysis/` — Core algorithm and analysis workflows

19.3.4 DSL Fundamentals: A Query Mental Model and Advanced Workflows: Compositional Queries Under Uncertainty (DSL)

- `examples/dsl_zoo/` — DSL-focused patterns and helper datasets

19.3.5 GUI Overview: Where It Helps, Where It Does Not (GUI)

- `gui/app.py` — Main application
- `gui/README.md` — Setup instructions

19.4 Key Scripts Reference

19.4.1 Makefile Targets

The `Makefile` provides common development tasks:

```
make setup           # Create virtual environment
make dev-install     # Install with dev dependencies
make test            # Run test suite
make test-coverage   # Generate coverage report
make lint            # Run code quality checks
make format          # Format code with black
make docs            # Build documentation
make clean           # Clean build artifacts
```

19.4.2 Development Scripts

```
scripts/
├── run_tests.py              # Test runner
├── generate_docs.py          # Documentation generator
└── benchmark.py              # Performance benchmarking
```

19.5 Configuration Files

19.5.1 Package Configuration

pyproject.toml — Modern Python package configuration (PEP 518):

- Package metadata (name, version, authors)
- Dependencies and extras (`[infomap]`, `[viz]`, etc.)
- Build system configuration
- Tool configurations (black, ruff, mypy)

19.5.2 Testing Configuration

pytest.ini — Test framework configuration:

- Test discovery patterns
- Coverage settings
- Markers for test categories

19.5.3 Documentation Configuration

docfiles/conf.py — Sphinx documentation configuration:

- Extensions (autodoc, napoleon, mathjax)
- Theme settings (sphinx_rtd_theme)
- Build options

19.5.4 Docker Configuration

Dockerfile — Container image definition **docker-compose.yml** — Multi-service orchestration

[Detailed Docker configs in Appendix B]

19.6 Data Files

19.6.1 Sample Datasets

```
datasets/
├── karate_multiplex.edgelist
├── example_biological.graphml
└── README.md
```

19.6.2 External Datasets

The `multilayer_datasets/` directory contains links and scripts for downloading larger datasets used in case studies.

19.7 Summary

Key directories:

- `py3plex/` — Main package code
- `tests/` — Test suite
- `examples/` — Examples mapped to book chapters (counts evolve with releases)
- `docfiles/` — Documentation source
- `gui/` — Web interface

Development workflow:

1. Clone repository
2. `make setup` to create environment
3. `make dev-install` to install dependencies
4. Modify code, run `make test`
5. Use `examples/` as reference

Finding code:

- Algorithms → `py3plex/algorithms/`
- DSL → `py3plex/dsl/`
- Examples → `examples/` (organized by topic)

CHAPTER

TWENTY

APPENDIX B: DOCKER, DOCKER COMPOSE, AND DEPLOYMENT

This appendix provides Docker configurations and deployment instructions for py3plex, including controlled-network deployment notes for the GUI.

20.1 Dockerfile Reference

20.1.1 Main Dockerfile

The repository includes a Dockerfile for containerized execution:

```
# File: Dockerfile
FROM python:3.10-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    git \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements
COPY pyproject.toml .
COPY README.md .

# Install py3plex
RUN pip install --no-cache-dir -e .

# Copy application code
COPY py3plex/ ./py3plex/
COPY examples/ ./examples/

# Set up entry point
ENTRYPOINT ["python", "-m", "py3plex"]
CMD ["--help"]
```

20.1.2 Building the Image

```
# Build image
docker build -t py3plex:latest .
```

```
# Build with specific Python version
docker build --build-arg PYTHON_VERSION=3.11 -t py3plex:3.11 .

# Build with version tag matching book release
docker build -t py3plex:1.1.5 .
```

20.1.3 Running Containers

```
# Run command
docker run --rm py3plex:latest --version

# Run analysis script with uv
docker run --rm py3plex:latest uv run examples/getting_started/01_basic_query.py

# Or using python directly
docker run --rm py3plex:latest python examples/getting_started/01_basic_query.py

# Mount data directory
docker run --rm -v $(pwd)/data:/data py3plex:latest python analysis.py

# Interactive shell
docker run --rm -it py3plex:latest /bin/bash
```

20.2 Docker Compose

Note

Docker Compose Command: This book uses the modern `docker compose` command (Docker CLI plugin, available since Docker 20.10). If you have an older Docker version, you may need to use the legacy `docker-compose` command instead. The configuration file is always named `docker-compose.yml` regardless of which command you use.

20.2.1 GUI Deployment (Experimental)

```
# File: docker-compose.yml
version: '3.8'

services:
  gui:
    build:
      context: .
      dockerfile: gui/Dockerfile
    ports:
      - "8000:8000"
    volumes:
      - ./data:/app/data
      - ./uploads:/app/uploads
    environment:
      - ENV=production
      - SECRET_KEY=${SECRET_KEY}
```

```
    restart: unless-stopped

  # Optional: nginx reverse proxy
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./ssl:/etc/nginx/ssl:ro
    depends_on:
      - gui
    restart: unless-stopped
```

20.2.2 Running with Docker Compose

```
# Start services
docker compose up -d

# View logs
docker compose logs -f gui

# Stop services
docker compose down

# Rebuild after changes
docker compose up -d --build
```

20.3 Controlled Environment Deployment

Warning

The GUI is experimental and not designed for public internet deployment. The configurations below are for controlled, trusted environments (for example, an internal lab network or VPN). They reduce risk but do not convert the GUI into a public-facing hardened service.

20.3.1 Security Hardening (Trusted Networks Only)

1. Use environment variables for secrets:

```
# .env file (DO NOT commit to git)
SECRET_KEY=your-strong-secret-key-here
DATABASE_URL=postgresql://user:pass@host/db
```

```
# docker-compose.yml
services:
  gui:
    env_file:
      - .env
```

2. Run as non-root user:

```
# Add to Dockerfile
RUN useradd -m -u 1000 py3plex
USER py3plex
```

3. Use read-only file systems where possible:

```
services:
  gui:
    read_only: true
    tmpfs:
      - /tmp
      - /var/tmp
```

20.3.2 Nginx Reverse Proxy

nginx.conf:

```
events {
    worker_connections 1024;
}

http {
    upstream gui {
        server gui:8000;
    }

    server {
        listen 80;
        server_name yourdomain.com;

        # Redirect to HTTPS
        return 301 https://$server_name$request_uri;
    }

    server {
        listen 443 ssl;
        server_name yourdomain.com;

        ssl_certificate /etc/nginx/ssl/cert.pem;
        ssl_certificate_key /etc/nginx/ssl/key.pem;

        # Security headers
        add_header X-Frame-Options "SAMEORIGIN";
        add_header X-Content-Type-Options "nosniff";
        add_header X-XSS-Protection "1; mode=block";

        location / {
            proxy_pass http://gui;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
```

```
        }

        # File upload limits
        client_max_body_size 100M;
    }
}
```

20.3.3 TLS/SSL Configuration

Using Let's Encrypt:

```
# Install certbot
docker run --rm -v $(pwd)/ssl:/etc/letsencrypt \
    certbot/certbot certonly --standalone \
    -d yourdomain.com \
    --email your@email.com \
    --agree-tos
```

Self-signed certificates (development only):

```
# Generate self-signed certificate
openssl req -x509 -newkey rsa:4096 \
    -keyout ssl/key.pem \
    -out ssl/cert.pem \
    -days 365 -nodes
```

20.3.4 Authentication Setup

Basic HTTP authentication (simple):

```
server {
    ...

    location / {
        auth_basic "Restricted Access";
        auth_basic_user_file /etc/nginx/.htpasswd;

        proxy_pass http://gui;
    }
}
```

```
# Generate .htpasswd file
htpasswd -c .htpasswd username
```

OAuth2 Proxy (advanced):

Use `oauth2-proxy` as an additional service for integration with GitHub, Google, etc.

20.4 Security Checklist

20.4.1 Before Public Deployment

- [ ] Use HTTPS (TLS/SSL) for all connections

- [ ] Set strong `SECRET_KEY` environment variable
- [ ] Enable authentication (HTTP Basic, OAuth2, etc.)
- [ ] Run containers as non-root user
- [ ] Use read-only file systems where possible
- [ ] Set appropriate file upload limits
- [ ] Enable security headers in nginx
- [ ] Regularly update base images (`docker pull python:3.10-slim`)
- [ ] Monitor logs for suspicious activity
- [ ] Implement rate limiting (via nginx or application)
- [ ] Sanitize user inputs (especially file uploads)
- [ ] Restrict file types for uploads
- [ ] Scan uploaded files for malware
- [ ] Use network isolation (Docker networks)
- [ ] Keep dependencies up to date (`pip install --upgrade`)

20.4.2 Monitoring

```yaml
# Add health checks to docker-compose.yml
services:
  gui:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
```

20.5 Cloud Deployment

20.5.1 AWS Deployment

Using ECS (Elastic Container Service):

1. Push image to ECR (Elastic Container Registry)
2. Create ECS task definition
3. Deploy to ECS cluster
4. Use ALB (Application Load Balancer) for routing

Using EC2:

1. Launch EC2 instance
2. Install Docker
3. Copy docker-compose.yml to instance
4. Run `docker compose up -d`

20.5.2 Azure Deployment

Using Azure Container Instances:

```
az container create \
    --resource-group myResourceGroup \
    --name py3plex-gui \
    --image py3plex:latest \
    --cpu 2 --memory 4 \
    --ports 8000
```

20.5.3 GCP Deployment

Using Cloud Run:

1. Push image to Google Container Registry
2. Deploy to Cloud Run
3. Configure domain and SSL

20.6 Performance Tuning

20.6.1 Container Resource Limits

```
services:
  gui:
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 4G
        reservations:
          cpus: '1.0'
          memory: 2G
```

20.6.2 Caching

Use Docker layer caching to speed up builds:

```
# Install dependencies first (cached layer)
COPY pyproject.toml .
RUN pip install -e .

# Copy code second (changes frequently)
COPY py3plex/ ./py3plex/
```

20.6.3 Multi-Stage Builds

For smaller production images:

```
# Build stage
FROM python:3.10 as builder
WORKDIR /app
```

```
COPY pyproject.toml .
RUN pip install --user -e .

# Runtime stage
FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY py3plex/ ./py3plex/
ENV PATH=/root/.local/bin:$PATH
```

20.7 Troubleshooting

20.7.1 Common Issues

Issue: Container exits immediately

Check logs:

```
docker logs <container_id>
```

Issue: Permission denied on mounted volumes

Fix permissions safely by matching container user to host user:

Method 1: Run container with your user ID (recommended)

```
docker run --user "$(id -u):$(id -g)" -v $(pwd)/data:/data py3plex-image
```

Method 2: Change ownership to match container user

```
# If container runs as user 1000:1000 (typical)
sudo chown -R 1000:1000 ./data
```

Method 3: Use docker compose with user mapping

```
services:
  py3plex:
    user: "${UID}:${GID}"
    volumes:
      - ./data:/data
```

Then run: `UID=$(id -u) GID=$(id -g) docker compose up`

Warning

Avoid `chmod -R 777 ./data` as it grants world-writable permissions and creates a security risk. Use user/group mapping instead.

Issue: Out of memory

Increase Docker memory limits or container resources.

20.8 Summary

This appendix provided:

- Complete Dockerfile and docker-compose.yml examples
- Security hardening guidelines
- Production deployment configurations (nginx, TLS, authentication)
- Cloud deployment options (AWS, Azure, GCP)
- Performance tuning strategies

Key recommendations for production:

1. Always use HTTPS
2. Enable authentication
3. Run as non-root
4. Monitor and log
5. Keep dependencies updated
6. Follow security checklist

See also: *GUI Overview: Where It Helps, Where It Does Not* (page 35) for high-level GUI overview

CHAPTER

TWENTYONE

APPENDIX C: DETAILED VALIDATION SCRIPTS

This appendix provides detailed validation strategies and test scripts for ensuring correctness of multilayer network algorithms. These complement the high-level overview in *Testing and Validation as Methodological Controls* (page 31).

21.1 Validation Philosophy

py3plex uses multiple validation strategies:

1. **Unit tests** — Test individual functions in isolation
2. **Property-based tests** — Test invariants that should always hold
3. **Reference runs** — Compare against known-good results
4. **Conservation tests** — Verify physical/mathematical laws (e.g., probability conservation)

21.2 Random Walk Validation

21.2.1 Conservation Tests

Random walks must conserve probability—the sum of transition probabilities from any state must equal 1:

```
# File: tests/test_random_walk_conservation.py
import pytest
import numpy as np
from py3plex.algorithms.paths import random_walk

def test_probability_conservation():
    """Verify random walk conserves probability."""
    network = create_test_network()

    # Run random walk
    walks = random_walk(network, num_walks=1000, walk_length=100)

    # Compute transition matrix
    trans_matrix = compute_transition_matrix(walks)

    # Check row sums = 1
    row_sums = trans_matrix.sum(axis=1)
    assert np.allclose(row_sums, 1.0, atol=1e-6)
```

21.2.2 Bias Validation

Node2Vec introduces biases (return parameter p, in-out parameter q). Validate that biases have the intended effect:

```
# File: tests/test_node2vec_bias.py
def test_return_bias():
    """Higher p should reduce probability of returning to previous node."""
    network = create_test_network()

    # Low p (encourage return)
    walks_low_p = node2vec_walk(network, p=0.25, q=1.0, num_walks=1000)
    return_rate_low = count_immediate_returns(walks_low_p)

    # High p (discourage return)
    walks_high_p = node2vec_walk(network, p=4.0, q=1.0, num_walks=1000)
    return_rate_high = count_immediate_returns(walks_high_p)

    assert return_rate_high < return_rate_low
```

21.2.3 Reference Runs

Store reference outputs for deterministic tests:

```
# File: tests/test_dynamics_reference_runs.py
REFERENCE_DATA = {
    'sir_small_graph_42': {
        'beta': 0.3,
        'gamma': 0.1,
        'steps': 100,
        'seed': 42,
        'expected_final_recovered': 0.75
    }
}

def test_sir_matches_reference():
    """Ensure SIR dynamics match reference run."""
    ref = REFERENCE_DATA['sir_small_graph_42']

    network = make_tiny_chain_graph()
    sir = SIRDynamics(network, beta=ref['beta'], gamma=ref['gamma'])
    sir.set_seed(ref['seed'])

    results = sir.run(steps=ref['steps'])
    final_recovered = results.get_measure('state_counts')[2][-1] / network.number_of_nodes()

    assert abs(final_recovered - ref['expected_final_recovered']) < 0.01
```

21.3 Community Detection Validation

21.3.1 Modularity Bounds

Modularity Q must be in range [-0.5, 1.0]:

```python
# File: tests/test_community_bounds.py
def test_modularity_bounds():
    """Modularity must be in valid range."""
    network = create_test_network()

    communities = multilayer_louvain.best_partition(network.core_network)
    Q = compute_modularity(network, communities)

    assert -0.5 <= Q <= 1.0
```

21.3.2 Partition Quality

Every node must be assigned to exactly one community:

```python
def test_partition_completeness():
    """Every node must be in exactly one community."""
    network = create_test_network()
    communities = multilayer_louvain.best_partition(network.core_network)

    # All nodes accounted for
    assert set(communities.keys()) == set(network.get_nodes())

    # No overlapping assignments (in non-overlapping methods)
    assert len(communities) == len(set(communities.values()))
```

21.3.3 Singleton Detection

Validate that isolated nodes form their own communities:

```python
def test_singleton_communities():
    """Isolated nodes should form singleton communities."""
    network = create_network_with_isolates()
    communities = multilayer_louvain.best_partition(network.core_network)

    isolates = [n for n in network.get_nodes() if network.core_network.degree(n) == 0]

    # Each isolate in its own community
    for node in isolates:
        community = communities[node]
        same_community = [n for n, c in communities.items() if c == community]
        assert same_community == [node]
```

21.4 Centrality Validation

21.4.1 Degree Centrality

Degree centrality should match actual degree counts:

```python
# File: tests/test_centrality_correctness.py
def test_degree_centrality():
    """Degree centrality should match computed degrees."""
    network = create_test_network()
```

```
    import networkx as nx
    degree_cent = nx.degree_centrality(network.core_network)

    for node in network.get_nodes():
        computed = degree_cent[node]
        expected = network.core_network.degree(node) / (network.number_of_nodes() - 1)
        assert abs(computed - expected) < 1e-10
```

21.4.2 Betweenness Centrality

Validate using known structures (e.g., star graphs):

```
def test_betweenness_star_graph():
    """In a star graph, center has betweenness 1.0."""
    network = create_star_graph(n=5)  # 1 center + 4 leaves

    import networkx as nx
    bc = nx.betweenness_centrality(network.core_network, normalized=True)

    center = ('center', 'layer')
    assert bc[center] == 1.0  # All paths go through center

    leaves = [n for n in network.get_nodes() if n != center]
    for leaf in leaves:
        assert bc[leaf] == 0.0  # Leaves are not on any shortest paths
```

21.5 Dynamics Validation

21.5.1 SIS Model Conservation

In SIS model, S(t) + I(t) = N for all t:

```
# File: tests/test_dynamics_conservation.py
def test_sis_conservation():
    """SIS model must conserve population."""
    network = create_test_network()
    N = network.number_of_nodes()

    sis = SISDynamics(network, beta=0.3, gamma=0.1)
    sis.set_seed(42)

    results = sis.run(steps=100)

    for t in range(100):
        S_t = results.get_measure('state_counts')[0][t]
        I_t = results.get_measure('state_counts')[1][t]
        assert S_t + I_t == N
```

21.5.2 SIR Model Conservation

In SIR model, S(t) + I(t) + R(t) = N for all t:

```
def test_sir_conservation():
    """SIR model must conserve population."""
    network = create_test_network()
    N = network.number_of_nodes()

    sir = SIRDynamics(network, beta=0.3, gamma=0.1)
    sir.set_seed(42)

    results = sir.run(steps=100)

    for t in range(100):
        S_t = results.get_measure('state_counts')[0][t]
        I_t = results.get_measure('state_counts')[1][t]
        R_t = results.get_measure('state_counts')[2][t]
        assert S_t + I_t + R_t == N
```

21.5.3 Steady State Validation

SIS model should reach steady state:

```
def test_sis_steady_state():
    """SIS should reach equilibrium."""
    network = create_test_network()

    sis = SISDynamics(network, beta=0.3, gamma=0.1)
    sis.set_seed(42)

    results = sis.run(steps=1000)
    prevalence = results.get_measure('prevalence')

    # Check last 100 steps are stable
    last_100 = prevalence[-100:]
    variance = np.var(last_100)
    assert variance < 0.01  # Low variance = steady state
```

21.6 Property-Based Testing

21.6.1 Using Hypothesis

Property-based tests use the `hypothesis` library to generate random inputs:

```
# File: tests/property/test_paths_properties.py
from hypothesis import given, strategies as st
from hypothesis import assume

@given(
    num_nodes=st.integers(min_value=3, max_value=20),
    num_layers=st.integers(min_value=1, max_value=5),
    seed=st.integers(min_value=0, max_value=1000)
)
```

```
def test_shortest_path_length_property(num_nodes, num_layers, seed):
    """Shortest path length should be <= graph diameter."""
    network = generate_random_network(num_nodes, num_layers, seed)
    assume(nx.is_connected(network.core_network))  # Skip disconnected graphs

    # Pick random source and target
    nodes = list(network.get_nodes())
    source, target = random.sample(nodes, 2)

    # Compute shortest path
    path = nx.shortest_path(network.core_network, source, target)
    path_length = len(path) - 1

    # Path length should be <= diameter
    diameter = nx.diameter(network.core_network)
    assert path_length <= diameter
```

21.6.2 Multilayer-Specific Properties

```
@given(num_nodes=st.integers(min_value=2, max_value=10))
def test_node_activity_bounds(num_nodes):
    """Node activity must be in [0, 1]."""
    network = generate_multiplex_network(num_nodes)

    for node_id in range(num_nodes):
        activity = mls.node_activity(network, str(node_id))
        assert 0.0 <= activity <= 1.0
```

21.7 Running Validation Tests

21.7.1 Full Test Suite

```
# Run all tests
pytest tests/

# Run only validation tests
pytest tests/ -k validation

# Run with coverage
pytest tests/ --cov=py3plex --cov-report=html
```

21.7.2 Property-Based Tests

```
# Run property tests (slower)
pytest tests/property/

# Run with more examples
pytest tests/property/ --hypothesis-profile=thorough
```

21.7.3 Continuous Validation

Tests run automatically on:

- Every commit (GitHub Actions)
- Pull requests
- Scheduled nightly runs

21.8 Summary

This appendix detailed validation strategies:

1. **Conservation tests** — Physical/mathematical laws must hold
2. **Reference runs** — Deterministic tests against known-good outputs
3. **Boundary tests** — Values must be in valid ranges
4. **Property-based tests** — Invariants hold for random inputs
5. **Structure tests** — Algorithmic correctness on known structures

Key test files:

- `tests/test_dynamics_reference_runs.py` — Reference dynamics data
- `tests/test_random_walk_conservation.py` — Probability conservation
- `tests/property/` — Property-based tests
- `tests/test_centrality_correctness.py` — Centrality validation

See also: *Testing and Validation as Methodological Controls* (page 31) for high-level testing overview

CHAPTER

TWENTYTWO

APPENDIX D: ERROR HANDLING AND EXCEPTION HIERARCHY

This appendix documents py3plex's exception classes and error handling conventions.

22.1 Exception Hierarchy

py3plex defines a hierarchy of exceptions for different error categories:

```
Py3plexException (base)
├── ValidationError
│   ├── InvalidNetworkError
│   ├── InvalidLayerError
│   └── InvalidNodeError
├── IOError
│   ├── FileFormatError
│   ├── FileNotFoundError
│   └── SerializationError
├── DSLError
│   ├── QuerySyntaxError
│   ├── QueryExecutionError
│   └── UnsupportedFeatureError
├── AlgorithmError
│   ├── ConvergenceError
│   └── InsufficientDataError
└── ConfigurationError
```

22.2 Base Exception

```python
class Py3plexException(Exception):
    """Base exception for all py3plex errors."""
    pass
```

All py3plex exceptions inherit from this base class, allowing users to catch any library error:

```python
from py3plex.exceptions import Py3plexException

try:
    result = some_py3plex_operation()
except Py3plexException as e:
    print(f"py3plex error: {e}")
```

22.3 Validation Errors

22.3.1 InvalidNetworkError

Raised when a network is in an invalid state:

```
from py3plex.exceptions import InvalidNetworkError

# Example: Empty network
if network.number_of_nodes() == 0:
    raise InvalidNetworkError("Cannot compute metrics on empty network")
```

Common causes:

- Empty network (no nodes or edges)
- Disconnected graph when algorithm requires connected
- Directed graph when undirected expected (or vice versa)

22.3.2 InvalidLayerError

Raised when referencing a nonexistent layer:

```
from py3plex.exceptions import InvalidLayerError

if layer_name not in network.get_layers():
    raise InvalidLayerError(f"Layer '{layer_name}' does not exist")
```

Common causes:

- Typo in layer name
- Layer removed after reference created
- Case sensitivity mismatch

22.3.3 InvalidNodeError

Raised when referencing a nonexistent node:

```
from py3plex.exceptions import InvalidNodeError

if node not in network.core_network:
    raise InvalidNodeError(f"Node {node} not found in network")
```

22.4 I/O Errors

22.4.1 FileFormatError

Raised when file format is invalid or unsupported:

```
from py3plex.exceptions import FileFormatError

try:
    network.load_network("data.xyz", input_type="xyz")
```

```
except FileFormatError as e:
    print(f"Unsupported format: {e}")
```

Common causes:

- Unrecognized file extension
- Malformed file content
- Missing required fields in CSV/JSON

22.4.2 SerializationError

Raised during read/write operations:

```
from py3plex.exceptions import SerializationError

try:
    write(network, "output.arrow")
except SerializationError as e:
    print(f"Failed to serialize: {e}")
```

22.5 DSL Errors

22.5.1 QuerySyntaxError

Raised for invalid DSL query syntax:

```
from py3plex.dsl import execute_query
from py3plex.exceptions import QuerySyntaxError

try:
    result = execute_query(network, 'SELECT nodes WHRE degree > 5')  # Typo
except QuerySyntaxError as e:
    print(f"Query syntax error: {e}")
    # Suggestion: "Did you mean 'WHERE'?"
```

Common causes:

- Typos in keywords (WHRE instead of WHERE)
- Missing quotes around layer names
- Invalid comparison operators

22.5.2 QueryExecutionError

Raised when query execution fails:

```
from py3plex.exceptions import QueryExecutionError

try:
    result = execute_query(network, 'SELECT nodes COMPUTE nonexistent_measure')
except QueryExecutionError as e:
    print(f"Query execution failed: {e}")
```

Common causes:

- Referencing nonexistent measures
- Applying measure to unsuitable graph (e.g., eigenvector centrality on disconnected graph)
- Resource limits exceeded

22.5.3 UnsupportedFeatureError

Raised when using experimental or unimplemented features:

```
from py3plex.exceptions import UnsupportedFeatureError

try:
    result = Q.edges().where(weight__gt=0.5).execute(network)
except UnsupportedFeatureError as e:
    print(f"Feature not yet supported: {e}")
```

22.6 Algorithm Errors

22.6.1 ConvergenceError

Raised when iterative algorithms fail to converge:

```
from py3plex.exceptions import ConvergenceError

try:
    centrality = compute_eigenvector_centrality(network, max_iter=100)
except ConvergenceError as e:
    print(f"Algorithm did not converge: {e}")
```

22.6.2 InsufficientDataError

Raised when there's not enough data for analysis:

```
from py3plex.exceptions import InsufficientDataError

if network.number_of_nodes() < 3:
    raise InsufficientDataError("Need at least 3 nodes for community detection")
```

22.7 Configuration Errors

22.7.1 ConfigurationError

Raised for invalid configuration or parameters:

```
from py3plex.exceptions import ConfigurationError

if beta <= 0 or gamma <= 0:
    raise ConfigurationError("SIR parameters beta and gamma must be positive")
```

22.8 Error Handling Best Practices

22.8.1 Catch Specific Exceptions

Prefer catching specific exceptions over generic ones:

```
# Good: Specific exception handling
from py3plex.exceptions import InvalidLayerError

try:
    result = Q.nodes().from_layers(L["social"]).execute(network)
except InvalidLayerError:
    print("Layer 'social' not found. Available layers:", network.get_layers())

# Avoid: Generic exception (too broad)
try:
    result = Q.nodes().from_layers(L["social"]).execute(network)
except Exception as e:  # Too generic
    print(f"Something went wrong: {e}")
```

22.8.2 Provide Context

Include helpful context in error messages:

```
# Good: Helpful context
if layer_name not in network.get_layers():
    available = ", ".join(network.get_layers())
    raise InvalidLayerError(
        f"Layer '{layer_name}' not found. Available layers: {available}"
    )

# Less helpful
if layer_name not in network.get_layers():
    raise InvalidLayerError(f"Invalid layer")
```

22.8.3 Validation at Entry Points

Validate inputs early:

```
def compute_centrality(network, measure='degree'):
    """Compute node centrality."""
    # Validate early
    if network.number_of_nodes() == 0:
        raise InvalidNetworkError("Cannot compute centrality on empty network")

    if measure not in ['degree', 'betweenness', 'closeness']:
        raise ValueError(f"Unknown measure: {measure}")

    # Proceed with computation
    ...
```

22.8.4 Clean Resource Cleanup

Use context managers for resources:

```
# Good: Automatic cleanup
from py3plex.io import read

try:
    graph = read('network.arrow')
    # Process graph
except IOError as e:
    print(f"Failed to read file: {e}")
finally:
    # Resources automatically cleaned up
```

22.9 Error Messages Guidelines

1. **Be specific:** State what went wrong and why
2. **Suggest fixes:** Offer alternatives when possible
3. **Include context:** Show relevant values (layer names, node counts, etc.)
4. **Be concise:** One sentence explanation, then details if needed

Examples:

```
# Good error message
raise InvalidLayerError(
    f"Layer 'socail' not found. Did you mean 'social'? "
    f"Available layers: {', '.join(network.get_layers())}"
)

# Good error message
raise InsufficientDataError(
    f"Community detection requires at least 3 nodes, but network has only "
    f"{network.number_of_nodes()}"
)

# Less helpful
raise InvalidLayerError("Layer error")  # Too vague
```

22.10 Logging

py3plex uses Python's logging module:

```
import logging

# Configure logging level
logging.basicConfig(level=logging.INFO)

# py3plex will log warnings and errors
network.load_network("data.edgelist")  # Logs any warnings
```

Available log levels:

- `DEBUG` — Detailed information for debugging
- `INFO` — General information
- `WARNING` — Recoverable issues
- `ERROR` — Serious problems
- `CRITICAL` — Fatal errors

22.11 Summary

Exception hierarchy:

- Base: `Py3plexException`
- Categories: Validation, I/O, DSL, Algorithm, Configuration

Best practices:

1. Catch specific exceptions
2. Provide helpful context in messages
3. Validate early
4. Suggest fixes when possible
5. Use logging for non-fatal issues

Example usage:

```
from py3plex.core import multinet
from py3plex.exceptions import InvalidLayerError, Py3plexException

try:
    network = multinet.multi_layer_network()
    network.load_network("data.edgelist")

    result = Q.nodes().from_layers(L["social"]).execute(network)

except InvalidLayerError as e:
    print(f"Layer not found: {e}")
except Py3plexException as e:
    print(f"py3plex error: {e}")
```

See also: *Advanced Workflows: Compositional Queries Under Uncertainty* (page 21) for error recovery strategies in workflows

22.12 Pedagogical Error Messages (DSL v2)

Py3plex DSL v2 implements Rust-style pedagogical error messages that help users understand and fix issues quickly. Errors include:

1. **What the user likely intended**
2. **Why the operation failed**
3. **1-2 corrected query examples**
4. **Common pitfall notes (when applicable)**

22.12.1 Error Philosophy

Guidance over Obscurity:

Traditional error messages say "what went wrong". Py3plex errors explain "what to do about it".

```
# Traditional error (not helpful)
Error: Invalid syntax

# Pedagogical error (helpful)
DslSyntaxError: Cannot call .where() after .end_grouping()

 You probably wanted to: Filter nodes before grouping

 Why this failed: After .end_grouping(), the query context
   returns to ungrouped state. The .where() clause filters the
   entire result, not individual groups.

 Corrected examples:
  1. Q.nodes().where(degree__gt=3).per_layer().top_k(5)
  2. Q.nodes().per_layer().filter_within_groups(degree__gt=3)

  Common pitfall: Grouping operations (.per_layer(), .group_by())
   create a new context. Apply filters before grouping for clarity.
```

Core Principles:

- **Actionable**: Always suggest a fix, never just complain
- **Contextual**: Explain *why* in terms of DSL/multilayer semantics
- **Educational**: Help users understand the underlying concepts
- **Concise**: Keep examples short and focused

22.12.2 Enhanced DSL Error Classes

DslSyntaxError — Enhanced with pedagogical context

Raised for invalid DSL syntax or query structure:

```
from py3plex.dsl.errors import DslSyntaxError

# Example: Invalid method chaining
try:
    Q.nodes().execute(net).where(degree__gt=3)  # Execute too early
except DslSyntaxError as e:
    print(e)
# Output:
# DslSyntaxError: Cannot call .where() on QueryResult
#
#  You probably wanted to: Filter before executing
#
#  Why this failed: .execute() returns a QueryResult object,
#    which is immutable. Query building methods like .where()
#    must be called before .execute().
#
#  Corrected examples:
```

```
#   1. Q.nodes().where(degree__gt=3).execute(net)
#   2. result = Q.nodes().execute(net); df = result.to_pandas().query("degree > 3")
```

DslExecutionError — Enhanced with runtime context

Raised when query execution fails due to runtime conditions:

```
from py3plex.dsl.errors import DslExecutionError

# Example: Missing required metric
try:
    Q.nodes().where(betweenness__gt=0.1).execute(net)  # Betweenness not computed
except DslExecutionError as e:
    print(e)
# Output:
# DslExecutionError: Cannot filter on 'betweenness' (not computed)
#
#  You probably wanted to: Compute betweenness before filtering
#
#  Why this failed: The .where() clause references 'betweenness',
#    but this metric hasn't been computed yet. Autocompute is
#    disabled for filters to prevent silent expensive operations.
#
#  Corrected examples:
#   1. Q.nodes().compute("betweenness_centrality").where(betweenness__gt=0.1)
#   2. Q.nodes().where(degree__gt=5).compute("betweenness_centrality")
#
#   Common pitfall: Some metrics (degree) are cheap and autocomputed.
#    Expensive metrics (betweenness, closeness) require explicit .compute()
```

MultilayerSemanticError — Multilayer-specific guidance

Raised for common multilayer network semantic issues:

```
from py3plex.dsl.errors import MultilayerSemanticError

# Example: Node replica confusion
try:
    # User expects 100 unique nodes, gets 300 node replicas
    result = Q.nodes().execute(multilayer_net)
    assert result.count == 100  # Fails if network has 3 layers
except AssertionError:
    raise MultilayerSemanticError(
        "Node count mismatch: Expected 100, got 300",
        semantic_issue="Counting node replicas instead of physical nodes",
        multilayer_context=(
            "In multilayer networks, each physical node appears in multiple "
            "layers as a 'node replica'. Q.nodes() returns ALL replicas, "
            "not unique physical nodes."
        ),
        examples=[
            "To count physical nodes: len(set(n[0] for n in result.items))",
            "To work per-layer: Q.nodes().per_layer().execute(net)",
            "To get single layer: Q.nodes().from_layers(L['social']).execute(net)"
        ]
```

```
    )
```

22.12.3 Example: Before and After

Before (traditional error):

```
AttributeError: 'QueryResult' object has no attribute 'coverage'

# User is confused: What's coverage? Where do I use it?
```

After (pedagogical error):

```
DslExecutionError: .coverage() requires active grouping

 You probably wanted to: Apply coverage after .per_layer()

 Why this failed: The .coverage() method filters items based
   on their presence across groups. You must create groups first
   with .per_layer() or .group_by().

 Corrected examples:
  1. Q.nodes().per_layer().top_k(5, "degree").end_grouping().coverage(mode="all")
  2. Q.nodes().from_layers(L["*"]).per_layer().compute("degree").coverage(mode="any")

  Common pitfall: .coverage() is a post-grouping filter, not a
   pre-grouping filter. It answers "which items appear across groups?"
```

22.12.4 Using Pedagogical Errors Effectively

As a user:

1. **Read the entire error** — Don't stop at the first line
2. **Try the corrected examples** — They're tested and known to work
3. **Learn the pitfall** — Understand the underlying concept
4. **Check documentation** — Errors reference relevant sections

As a developer:

When raising DSL errors, provide rich context:

```
from py3plex.dsl.errors import DslExecutionError

# Good: Rich pedagogical error
raise DslExecutionError(
    f"Cannot filter on '{field}' (not computed)",
    intent=f"Compute {field} before filtering",
    why_failed=(
        f"The .where() clause references '{field}', but this metric "
        f"hasn't been computed yet. Autocompute is disabled for filters."
    ),
    examples=[
        f"Q.nodes().compute('{field}').where({field}__gt=threshold)",
        f"Q.nodes().where(degree__gt=5).compute('{field}')"  # Filter first
```

```
    ],
    pitfall=(
        "Some metrics (degree) are cheap and autocomputed. "
        "Expensive metrics require explicit .compute()"
    )
)
```

22.12.5 Error Recovery Patterns

Pattern 1: Inspect-Fix-Retry

```
try:
    result = Q.nodes().where(betweenness__gt=0.1).execute(net)
except DslExecutionError as e:
    print(e)  # Read pedagogical message

    # Apply suggested fix
    result = (
        Q.nodes()
         .compute("betweenness_centrality")
         .where(betweenness__gt=0.1)
         .execute(net)
    )
```

Pattern 2: Catch-and-Adapt

```
from py3plex.dsl.errors import MultilayerSemanticError

try:
    # Assume single-layer semantics
    result = Q.nodes().compute("degree").execute(net)
    avg_degree = sum(result.attributes["degree"].values()) / result.count
except (KeyError, MultilayerSemanticError):
    # Adapt to multilayer semantics
    result = (
        Q.nodes()
         .per_layer()
         .compute("degree")
         .aggregate("mean")  # Per-layer averages
         .execute(net)
    )
```

Pattern 3: Defensive Query Building

```
# Check capabilities before building complex queries
if len(net.get_layers()) > 1:
    # Multilayer network - use layer-aware query
    result = (
        Q.nodes()
         .per_layer()
         .compute("degree")
         .top_k(10, "degree")
         .end_grouping()
         .coverage(mode="any")  # Nodes in any layer
```

```
        .execute(net)
    )
else:
    # Single-layer network - use simple query
    result = (
        Q.nodes()
         .compute("degree")
         .order_by("-degree")
         .limit(10)
         .execute(net)
    )
```

22.12.6 Summary: Error Handling Philosophy

Traditional approach:

- Error: "What went wrong"
- User: Googles error message
- Result: Frustrated user, support ticket

Py3plex pedagogical approach:

- Error: "What went wrong" + "Why" + "How to fix" + "Learn more"
- User: Reads error, applies fix
- Result: Self-sufficient user, learned a concept

Key benefits:

1. **Faster debugging**: Users fix issues without leaving their editor
2. **Better learning**: Errors teach multilayer network concepts
3. **Reduced support**: Common mistakes are explained inline
4. **Higher quality code**: Users learn best practices from error messages

See also:

- DSL Builder API (Chapter 9) for query construction patterns
- Advanced Queries (Chapter 10) for error recovery in workflows
- Multilayer Basics (Chapter 2) for core multilayer network concepts

CHAPTER

TWENTYTHREE

APPENDIX E: EXTENDED API AND DSL REFERENCE

This appendix provides a quick reference for the py3plex API and DSL syntax.

23.1 Core API Reference

23.1.1 Creating Networks

```
from py3plex.core import multinet

# Create empty network
network = multinet.multi_layer_network(
    directed=False,           # True for directed networks
    network_type='multilayer' # or 'multiplex'
)
```

23.1.2 Adding Nodes and Edges

```
# Add edges (nodes created implicitly)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
], input_type="list")

# Add edges with attributes
network.add_edges([{
    'source': 'Alice',
    'source_type': 'friends',
    'target': 'Bob',
    'target_type': 'friends',
    'weight': 0.8,
    'timestamp': '2023-01-15'
}])
```

23.1.3 Loading Networks

```
# From file
network.load_network("data.edgelist", input_type="edgelist")

# From NetworkX
import networkx as nx
```

```
G = nx.karate_club_graph()
network.load_network_from_networkx(G, layer_name='social')
```

23.1.4 Basic Operations

```
# Get network information
layers = network.get_layers()
nodes = list(network.get_nodes())
edges = list(network.get_edges())

# Statistics
network.basic_stats()
num_nodes = network.get_number_of_nodes()
num_edges = network.get_number_of_edges()

# Layer-specific
nodes_per_layer = network.get_number_of_nodes_per_layer()
layer_subgraph = network.get_layer_subgraph('friends')
```

23.2 Algorithms API

23.2.1 Statistics

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density
density = mls.layer_density(network, 'friends')

# Node activity (fraction of layers a node appears in)
activity = mls.node_activity(network, 'Alice')

# Layer overlap
overlap = mls.layer_overlap(network, 'friends', 'colleagues')
```

23.2.2 Community Detection

```
from py3plex.algorithms.community_detection import multilayer_louvain

# Detect communities
communities = multilayer_louvain.best_partition(network.core_network)

# communities = {('Alice', 'friends'): 0, ('Bob', 'friends'): 0, ...}
```

23.2.3 Centrality

```
import networkx as nx

# Standard centralities (use NetworkX)
G = network.core_network
degree_cent = nx.degree_centrality(G)
```

```
betweenness = nx.betweenness_centrality(G)
closeness = nx.closeness_centrality(G)

# Explainable centrality (py3plex-specific)
from py3plex.algorithms.centrality.explain import explain_node_centrality

explanation = explain_node_centrality(
    network,
    node='Alice',
    measure='degree'
)
# Returns: {'score': 4, 'layer_breakdown': {...}, ...}
```

23.2.4 Dynamics

```
from py3plex.dynamics.models import SIRDynamics, SISDynamics

# Configure SIR model
sir = SIRDynamics(
    network,
    beta=0.3,      # Infection rate
    gamma=0.1,     # Recovery rate
    initial_infected={'Alice': 0}  # Initial conditions
)

# Set seed for reproducibility
sir.set_seed(42)

# Run simulation
results = sir.run(steps=100)

# Extract measures
prevalence = results.get_measure('prevalence')
recovered = results.get_measure('state_counts')[2]  # State 2 = recovered
```

23.3 DSL Quick Reference

23.3.1 Builder API (Recommended)

```
from py3plex.dsl import Q, L, Param

# Basic query
result = Q.nodes().execute(network)

# Filter by layer
result = Q.nodes().from_layers(L["social"]).execute(network)

# Filter by condition
result = Q.nodes().where(degree__gt=5).execute(network)

# Multiple conditions
```

```
result = Q.nodes().where(layer="social", degree__gt=3).execute(network)

# Layer algebra
result = Q.nodes().from_layers(L["social"] + L["work"]).execute(network)      # Union
result = Q.nodes().from_layers(L["social"] - L["bots"]).execute(network)     # Difference
result = Q.nodes().from_layers(L["social"] & L["work"]).execute(network)     # Intersection

# Compute measures
result = Q.nodes().compute("degree", "betweenness_centrality").execute(network)

# Order and limit
result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")
     .limit(10)
     .execute(network)
)

# Export results
result = (
    Q.nodes()
     .compute("degree")
     .execute(network)
)
result.to_pandas().to_csv("output.csv", index=False)

# Parameterized queries
result = (
    Q.nodes()
     .where(degree__gt=Param('min_degree'))
     .execute(network, params={'min_degree': 5})
)
```

23.3.2 String DSL Syntax

```
from py3plex.dsl import execute_query

# Basic syntax: SELECT target WHERE conditions COMPUTE measures

# Select all nodes
execute_query(network, 'SELECT nodes')

# Filter by layer
execute_query(network, 'SELECT nodes WHERE layer="social"')

# Filter by degree
execute_query(network, 'SELECT nodes WHERE degree > 5')

# Multiple conditions
execute_query(network, 'SELECT nodes WHERE layer="social" AND degree > 3')

# Compute measures
```

```
execute_query(network, 'SELECT nodes COMPUTE betweenness_centrality')

# Combined
execute_query(network,
    'SELECT nodes WHERE layer="social" AND degree > 3 '
    'COMPUTE degree betweenness_centrality'
)
```

23.3.3 DSL Operators

Comparison operators:

- = — Equal to
- != — Not equal to
- > — Greater than
- < — Less than
- >= — Greater than or equal
- <= — Less than or equal

Logical operators:

- `AND` — Logical AND
- `OR` — Logical OR
- `NOT` — Logical NOT

Filter modifiers (Builder API):

- `field=value` — Equal
- `field__ne=value` — Not equal
- `field__gt=value` — Greater than
- `field__gte=value` — Greater or equal
- `field__lt=value` — Less than
- `field__lte=value` — Less or equal

23.3.4 Available Measures

- `degree` — Node degree
- `degree_centrality` — Normalized degree
- `betweenness_centrality` — Betweenness
- `closeness_centrality` — Closeness
- `eigenvector_centrality` — Eigenvector
- `pagerank` — PageRank
- `clustering` — Clustering coefficient

23.3.5 Working with Results

```
result = Q.nodes().compute("degree").execute(network)

# Iteration
for node in result:
    print(node)

# Count
print(f"Found {result.count} nodes")

# Measures
degrees = result.measures['degree']
for node, degree in degrees.items():
    print(f"{node}: {degree}")

# To pandas
df = result.to_pandas()
```

23.4 I/O API

23.4.1 Modern I/O System

```
from py3plex.io import read, write, MultiLayerGraph

# Read (auto-detects format from extension)
graph = read('network.json')
graph = read('network.arrow')
graph = read('network.parquet')

# Write
write(network, 'output.json')
write(network, 'output.arrow')
write(network, 'output.parquet')

# Specify format explicitly
graph = read('file.dat', format='json')
write(network, 'file.dat', format='arrow')
```

23.4.2 Supported Formats

- **JSON** — `.json` — Human-readable
- **JSONL** — `.jsonl` — Streaming
- **CSV** — `.csv` — Spreadsheet-compatible
- **Arrow** — `.arrow` — High-performance
- **Parquet** — `.parquet` — Compressed

23.5 Visualization API

23.5.1 Basic Visualization

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Simple visualization
draw_multilayer_default(network.get_layers(), display=True)

# With layout options
draw_multilayer_default(
    network.get_layers(),
    display=True,
    layout='force_directed',
    node_size=300,
    edge_width=2
)
```

23.5.2 Matrix Visualization

```
# Visualize supra-adjacency matrix
network.visualize_matrix({"display": True})
```

23.6 Command-Line Interface

23.6.1 Basic Usage

```
# Show version
python -m py3plex --version

# Run self-test
python -m py3plex selftest

# Analyze network
python -m py3plex analyze network.edgelist

# Community detection
python -m py3plex communities network.edgelist --algorithm louvain
```

23.7 Summary

This appendix provided quick references for:

- Core API (creating networks, adding nodes/edges)
- Algorithms API (statistics, communities, centrality, dynamics)
- DSL syntax (both Builder API and string syntax)
- I/O API (reading/writing various formats)
- Visualization API
- Command-line interface

For detailed documentation:

- Main text chapters for concepts and examples
- Appendix A for repository layout
- Appendix D for error handling
- Online API docs: https://skblaz.github.io/py3plex/

CHAPTER

TWENTYFOUR

BIBLIOGRAPHY

This section provides references for further reading on multilayer networks and py3plex.

24.1 How to Cite This Book

Use the following citation for this manuscript edition:

```
Škrlj, B. (2025). Practical Multilayer Network Analysis with Py3plex (Version 1.1.5).
https://github.com/SkBlaz/py3plex
```

BibTeX:

```
@book{Skrlj2025book,
  author = {Škrlj, Blaž},
  title = {Practical Multilayer Network Analysis with Py3plex},
  year = {2025},
  version = {1.1.5},
  url = {https://github.com/SkBlaz/py3plex}
}
```

24.2 Core References

24.3 Py3plex Publications

24.4 Community Detection

24.5 Network Embeddings

24.6 Dynamics and Processes

24.7 Applications

24.7.1 Biological Networks

24.7.2 Social Networks

24.7.3 Transportation Networks

24.8 Software and Tools

24.9 Online Resources

Py3plex:

- Documentation: https://skblaz.github.io/py3plex/
- GitHub: https://github.com/SkBlaz/py3plex
- PyPI: https://pypi.org/project/py3plex/

Community:

- Network Science community: https://netscisociety.net/
- Complex Networks: https://www.complexnetworks.org/

Datasets:

- Network Repository: https://networkrepository.com/
- SNAP: Stanford Network Analysis Project: https://snap.stanford.edu/data/
- Netzschleuder: https://networks.skewed.de/

24.10 Recommended Reading Path

24.10.1 For Graduate Students

1. Start with [Kivelä2014] for comprehensive foundations
2. Read [Skrlj2019a] for py3plex overview
3. Read [Mucha2010] for community detection
4. Explore [Bianconi2018] textbook for deeper theory

24.10.2 For Practitioners

1. Start with [Skrlj2019a] for py3plex capabilities
2. Skim [Kivelä2014] for key concepts
3. Focus on application papers relevant to your domain
4. Use this book and online docs for implementation

24.10.3 For Researchers

1. Read [Kivelä2014] and [Boccaletti2014] thoroughly
2. Study [Domenico2013] for mathematical rigor
3. Read domain-specific application papers
4. Explore [Bianconi2018] for advanced topics

BIBLIOGRAPHY

[Kivelä2014] Kivelä, M., Arenas, A., Barthelemy, M., Gleeson, J. P., Moreno, Y., & Porter, M. A. (2014). Multilayer networks. *Journal of Complex Networks*, 2(3), 203-271. https://doi.org/10.1093/comnet/cnu016

Comprehensive review of multilayer network theory, mathematics, and applications. Essential reading.

[Boccaletti2014] Boccaletti, S., Bianconi, G., Criado, R., Del Genio, C. I., Gómez-Gardenes, J., Romance, M., … & Zanin, M. (2014). The structure and dynamics of multilayer networks. *Physics Reports*, 544(1), 1-122. https://doi.org/10.1016/j.physrep.2014.07.001

Physics-focused perspective on multilayer networks with emphasis on dynamics.

[Bianconi2018] Bianconi, G. (2018). *Multilayer Networks: Structure and Function*. Oxford University Press.

Comprehensive textbook covering theory, methods, and applications.

[Domenico2013] De Domenico, M., Solé-Ribalta, A., Cozzo, E., Kivelä, M., Moreno, Y., Porter, M. A., … & Arenas, A. (2013). Mathematical formulation of multilayer networks. *Physical Review X*, 3(4), 041022. https://doi.org/10.1103/PhysRevX.3.041022

Rigorous mathematical formulation of multilayer network representations.

[Skrlj2019a] Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. https://doi.org/10.1007/s41109-019-0203-7

Primary py3plex paper. Cite this when using py3plex in research.

[Skrlj2019b] Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex: A library for scalable multilayer network analysis and visualization. In *International Conference on Complex Networks and Their Applications* (pp. 757-768). Springer. https://doi.org/10.1007/978-3-030-05411-3_60

Conference paper focusing on visualization and scalability.

[Blondel2008] Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Louvain algorithm for community detection.

[Rosvall2008] Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences*, 105(4), 1118-1123. https://doi.org/10.1073/pnas.0706851105

Infomap algorithm for community detection.

[Mucha2010] Mucha, P. J., Richardson, T., Macon, K., Porter, M. A., & Onnela, J. P. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878. https://doi.org/10.1126/science.1184819

Multilayer community detection with modularity optimization.

[Grover2016] Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 855-864). https://doi.org/10.1145/2939672.2939754

Node2Vec algorithm for network embeddings.

[Perozzi2014] Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 701-710). https://doi.org/10.1145/2623330.2623732

DeepWalk algorithm for learning node representations.

[Pastor-Satorras2015] Pastor-Satorras, R., Castellano, C., Van Mieghem, P., & Vespignani, A. (2015). Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3), 925. https://doi.org/10.1103/RevModPhys.87.925

Comprehensive review of epidemic modeling on networks.

[Buldyrev2010] Buldyrev, S. V., Parshani, R., Paul, G., Stanley, H. E., & Havlin, S. (2010). Catastrophic cascade of failures in interdependent networks. *Nature*, 464(7291), 1025-1028. https://doi.org/10.1038/nature08932

Cascading failures in interdependent networks.

[Gligorijevic2019] Gligorijević, V., Barot, M., & Bonneau, R. (2019). deepNF: deep network fusion for protein function prediction. *Bioinformatics*, 35(5), 757-766. https://doi.org/10.1093/bioinformatics/bty440

Application to biological network analysis.

[Magnani2013] Magnani, M., & Rossi, L. (2013). Pareto distance for multilayer network analysis. In *Social Computing, Behavioral-Cultural Modeling and Prediction* (pp. 249-256). Springer. https://doi.org/10.1007/978-3-642-37210-0_27

Social network analysis with multilayer methods.

[Gallotti2014] Gallotti, R., & Barthelemy, M. (2014). Anatomy and efficiency of urban multimodal mobility. *Scientific Reports*, 4(1), 6911. https://doi.org/10.1038/srep06911

Transportation networks as multilayer systems.

[NetworkX] Hagberg, A., Swart, P., & Schult, D. A. (2008). Exploring network structure, dynamics, and function using NetworkX. *Los Alamos National Lab.(LANL), Los Alamos, NM (United States)*.

NetworkX documentation: https://networkx.org/

[Arrow] Apache Arrow Development Team. (2023). Apache Arrow. https://arrow.apache.org/

High-performance columnar data format.